OP-TEE 디버깅 및 트러블슈팅 가이드

이 문서는 OP-TEE 개발 과정에서 자주 발생하는 문제들과 해결 방법을 상세히 설명합니다. 실제 개발 경험을 바탕으로 한 실용적인 디버깅 기법들을 제공합니다.

1. 개발 환경 설정 문제

1.1 빌드 환경 문제

문제: 크로스 컴파일 환경에서 빌드 실패

```
# 일반적인 에러 메시지
arm-linux-gnueabihf-gcc: command not found
make: *** [all] Error 127
```

해결 방법:

```
# 1단계: 툴체인 설치 확인
which arm-linux-gnueabihf-gcc
echo $PATH

# 2단계: 툴체인 설치 (Ubuntu/Debian)
sudo apt-get update
sudo apt-get install gcc-arm-linux-gnueabihf
sudo apt-get install gcc-aarch64-linux-gnu # ARM64용
```

```
# 3단계: 환경 변수 설정
export CROSS_COMPILE=arm-linux-gnueabihf-
export ARCH=arm

# 4단계: 빌드 재시도
make clean
make -j$(nproc)
```

고급 디버깅:

```
# 컴파일러 버전 확인
arm-linux-gnueabihf-gcc --version

# 링커 문제 디버깅
arm-linux-gnueabihf-ld --verbose

# 라이브러리 경로 확인
arm-linux-gnueabihf-gcc -print-search-dirs
```

1.2 의존성 문제

문제: 필수 라이브러리 누락

```
# 에러 예시
```

```
fatal error: tee_internal_api.h: No such file or directory
```

해결 방법:

```
# 1단계: OP-TEE 소스 트리 구조 확인
ls -la optee/
├── build/
├── optee_os/
├── optee_client/
├── optee_test/
└── toolchains/

# 2단계: 환경 변수 설정
export OPTEE_DIR=/path/to/optee
export TA_DEV_KIT_DIR=$OPTEE_DIR/optee_os/out/arm/export-ta_arm32
export TEEC_EXPORT=$OPTEE_DIR/optee_client/out/export

# 3단계: 헤더 파일 경로 확인
find $OPTEE_DIR -name "tee_internal_api.h" -type f

# 4단계: Makefile 수정
CFLAGS += -I$(TA_DEV_KIT_DIR)/include
CFLAGS += -I$(TEEC_EXPORT)/include
```

2. TA 개발 문제

2.1 TA 로딩 실패

문제: TA가 로드되지 않음

```
// 클라이언트에서 발생하는 에러
TEEC_OpenSession failed: 0xFFFF0008 (TEE_ERROR_ITEM_NOT_FOUND)
```

디버깅 단계:

```
# 1단계: TA 파일 존재 확인
ls -la /lib/optee_armtz/
# 또는
ls -la /usr/lib/optee_armtz/

# 2단계: TA UUID 확인
hexdump -C your_ta.ta | head -20

# 3단계: OP-TEE 로그 확인
dmesg | grep -i optee
journalctl -u tee-supplicant

# 4단계: 상세 로그 활성화
echo 4 > /sys/module/optee/parameters/debug
```

TA 서명 문제:

```
# 서명 키 생성
openssl genrsa -out ta_key.pem 2048
openssl rsa -in ta_key.pem -pubout -out ta_pub_key.pem

# TA 서명
python3 sign_ta.py --key ta_key.pem --uuid $TA_UUID --in ta.elf --out ta.ta

# 서명 검증
python3 verify_ta.py --key ta_pub_key.pem --ta ta.ta
```

2.2 메모리 관리 문제

문제: 메모리 누수 및 힙 오버플로우

```
// 일반적인 메모리 문제 패턴
void *ptr = TEE_Malloc(size, 0);
// TEE_Free(ptr); <- 누락된 해제

// 버퍼 오버플로우
char buffer[100];
TEE_MemMove(buffer, src, 200); // 위험!
```

디버깅 기법:

```
// 1. 메모리 할당 추적 매크로
#ifdef DEBUG_MEMORY
#define DEBUG_MALLOC(size) \
do { \
void *ptr = TEE_Malloc(size, 0); \
DMSG("MALLOC: %p, size: %zu", ptr, size); \
return ptr; \
} while(0)

#define DEBUG_FREE(ptr) \
do { \
DMSG("FREE: %p", ptr); \
TEE_Free(ptr); \
ptr = NULL; \
} while(0)
#endif

// 2. 스택 오버플로우 검사
static void check_stack_usage(void)
{
volatile char stack_marker;
uintptr_t current_sp = (uintptr_t)&stack_marker;
```

```
uintptr_t stack_base = /* 스택 베이스 주소 */;

size_t stack_used = stack_base - current_sp;

if (stack_used > STACK_WARNING_THRESHOLD) {

EMSG("Stack usage warning: %zu bytes", stack_used);

}

}



// 3. 힙 상태 모니터링

static void dump_heap_stats(void)

{

struct malloc_stats stats;

get_malloc_stats(&stats);

DMSG("Heap stats:");

DMSG(" Total allocated: %zu bytes", stats.total_allocated);

DMSG(" Peak usage: %zu bytes", stats.peak_usage);

DMSG(" Free blocks: %u", stats.free_blocks);

}
```

2.3 암호화 연산 문제

문제: 암호화 연산 실패

```
// 일반적인 에러

TEE_Result res = TEE_CipherInit(operation, iv, iv_len);
```

```
if (res != TEE_SUCCESS) {
EMSG("Cipher init failed: 0x%x", res);
}
```

디버깅 접근법:

```
// 1. 알고리즘 지원 확인
static TEE_Result check_crypto_support(void)
{
TEE_Result res = TEE_SUCCESS;
// AES 지원 확인
if (!TEE_IsAlgorithmSupported(TEE_ALG_AES_ECB_NOPAD, TEE_ECC_CURVE_NONE)) {
EMSG("AES ECB not supported");
res = TEE_ERROR_NOT_SUPPORTED;
}
// RSA 지원 확인
if (!TEE_IsAlgorithmSupported(TEE_ALG_RSASSA_PKCS1_V1_5_SHA256, TEE_ECC_CURVE_NO
EMSG("RSA PKCS1 not supported");
res = TEE_ERROR_NOT_SUPPORTED;
}
return res;
}

// 2. 키 속성 검증
```

```
static void dump_key_attributes(TEE_ObjectHandle key)
{
TEE_Result res = TEE_SUCCESS;
uint32_t key_size = 0;
uint32_t key_usage = 0;
uint32_t algorithm = 0;
res = TEE_GetObjectBufferAttribute(key, TEE_ATTR_SECRET_VALUE,
NULL, &key_size);
DMSG("Key size: %u bits", key_size * 8);
TEE_GetObjectValueAttribute(key, TEE_ATTR_KEY_USAGE, &key_usage);
DMSG("Key usage: 0x%x", key_usage);
TEE_GetObjectValueAttribute(key, TEE_ATTR_ALGORITHM, &algorithm);
DMSG("Algorithm: 0x%x", algorithm);
}


// 3. 암호화 데이터 검증
static void dump_crypto_data(const char *label, const void *data, size_t len)
{
const uint8_t *bytes = (const uint8_t *)data;
char hex_str[256];
size_t hex_len = 0;
DMSG("%s (%zu bytes):", label, len);
for (size_t i = 0; i < MIN(len, 32); i++) {
```

```
hex_len += snprintf(hex_str + hex_len, sizeof(hex_str) - hex_len,
"%02x ", bytes[i]);
}
if (len > 32) {
snprintf(hex_str + hex_len, sizeof(hex_str) - hex_len, "...");
}
DMSG(" %s", hex_str);
}
```

3. 시스템 콜 디버깅

3.1 시스템 콜 추적

시스템 콜 로깅 활성화:

```
// core/kernel/tee_svc.c에 추가
#ifdef CFG_SYSCALL_TRACE
static void trace_syscall_entry(size_t num, unsigned long a0, unsigned long a1,
unsigned long a2, unsigned long a3)
{
const char *syscall_names[] = {
"sys_return", "log", "panic", "get_property",
"get_property_name_to_index", "open_ta_session",
"close_ta_session", "invoke_ta_command",
```

```
// ... 더 많은 시스템 콜 이름들
};
if (num < ARRAY_SIZE(syscall_names)) {
DMSG("SYSCALL[%zu]: %s(0x%lx, 0x%lx, 0x%lx, 0x%lx)",
num, syscall_names[num], a0, a1, a2, a3);
} else {
DMSG("SYSCALL[%zu]: unknown(0x%lx, 0x%lx, 0x%lx, 0x%lx)",
num, a0, a1, a2, a3);
}
}

static void trace_syscall_exit(size_t num, TEE_Result res)
{
DMSG("SYSCALL[%zu]: return 0x%x", num, res);
}
#endif
```

3.2 매개변수 검증 문제

문제: 잘못된 매개변수로 인한 시스템 콜 실패

```
// 매개변수 검증 강화
static TEE_Result validate_syscall_params(struct ts_session *sess,
unsigned long param_addr,
```

```
size_t param_size)
{
struct user_ta_ctx *utc = to_user_ta_ctx(sess->ctx);
// 1. NULL 포인터 검사
if (!param_addr && param_size > 0) {
EMSG("NULL pointer with non-zero size");
return TEE_ERROR_BAD_PARAMETERS;
}
// 2. 주소 범위 검사
if (!vm_buf_is_inside_um_private(&utc->uctx, (void *)param_addr, param_size)) {
EMSG("Parameter outside TA memory: 0x%lx (size: %zu)",
param_addr, param_size);
return TEE_ERROR_ACCESS_DENIED;
}
// 3. 정렬 검사
if (param_addr & (sizeof(void *) - 1)) {
EMSG("Unaligned parameter address: 0x%lx", param_addr);
return TEE_ERROR_BAD_PARAMETERS;
}
// 4. 크기 제한 검사
if (param_size > MAX_SYSCALL_PARAM_SIZE) {
EMSG("Parameter too large: %zu bytes", param_size);
return TEE_ERROR_BAD_PARAMETERS;
```

```
}
return TEE_SUCCESS;
}
```

4. 성능 문제 디버깅

4.1 성능 프로파일링

시간 측정 유틸리티:

```
// 고정밀 시간 측정
static uint64_t get_time_us(void)
{
return read_cntpct() / (read_cntfrq() / 1000000);
}

// 함수 실행 시간 측정 매크로
#define MEASURE_TIME(func_call) \
do { \
uint64_t start_time = get_time_us(); \
func_call; \
uint64_t end_time = get_time_us(); \
DMSG(#func_call " took %llu us", end_time - start_time); \
} while(0)
```

```
// 성능 카운터 구조체
struct perf_counter {
const char *name;
uint64_t total_time;
uint64_t call_count;
uint64_t min_time;
uint64_t max_time;
};

static struct perf_counter perf_counters[MAX_PERF_COUNTERS];

// 성능 카운터 업데이트
static void update_perf_counter(int counter_id, uint64_t elapsed_time)
{
struct perf_counter *pc = &perf_counters[counter_id];
pc->total_time += elapsed_time;
pc->call_count++;
if (elapsed_time < pc->min_time || pc->min_time == 0) {
pc->min_time = elapsed_time;
}
if (elapsed_time > pc->max_time) {
pc->max_time = elapsed_time;
```

```
}
}

// 성능 통계 출력
static void dump_perf_stats(void)
{
DMSG("=== Performance Statistics ===");
for (int i = 0; i < MAX_PERF_COUNTERS; i++) {
struct perf_counter *pc = &perf_counters[i];
if (pc->call_count > 0) {
uint64_t avg_time = pc->total_time / pc->call_count;
DMSG("%s:", pc->name);
DMSG(" Calls: %llu", pc->call_count);
DMSG(" Total: %llu us", pc->total_time);
DMSG(" Average: %llu us", avg_time);
DMSG(" Min: %llu us", pc->min_time);
DMSG(" Max: %llu us", pc->max_time);
}
}
}
```

4.2 메모리 사용량 모니터링

```
// 메모리 사용량 추적
struct memory_tracker {
size_t current_usage;
size_t peak_usage;
size_t allocation_count;
size_t free_count;
};

static struct memory_tracker mem_tracker = {0};

// 메모리 할당 래퍼
void *tracked_malloc(size_t size)
{
void *ptr = TEE_Malloc(size, 0);
if (ptr) {
mem_tracker.current_usage += size;
mem_tracker.allocation_count++;
if (mem_tracker.current_usage > mem_tracker.peak_usage) {
mem_tracker.peak_usage = mem_tracker.current_usage;
}
// 메모리 사용량 경고
if (mem_tracker.current_usage > MEMORY_WARNING_THRESHOLD) {
```

```
EMSG("High memory usage: %zu bytes", mem_tracker.current_usage);

}

}

return ptr;

}



// 메모리 해제 래퍼

void tracked_free(void *ptr, size_t size)

{

if (ptr) {

TEE_Free(ptr);

mem_tracker.current_usage -= size;

mem_tracker.free_count++;

}

}



// 메모리 누수 검사

static void check_memory_leaks(void)

{

if (mem_tracker.allocation_count != mem_tracker.free_count) {

EMSG("Memory leak detected:");

EMSG(" Allocations: %zu", mem_tracker.allocation_count);

EMSG(" Frees: %zu", mem_tracker.free_count);
```

```
EMSG(" Leaked: %zu",
mem_tracker.allocation_count - mem_tracker.free_count);
}
if (mem_tracker.current_usage > 0) {
EMSG("Memory still in use: %zu bytes", mem_tracker.current_usage);
}
}
```

5. 동시성 문제 디버깅

5.1 데드락 검출

```
// 뮤텍스 소유권 추적
struct mutex_info {
struct mutex *mutex;
uint32_t owner_thread;
const char *file;
int line;
uint64_t acquire_time;
};

static struct mutex_info mutex_table[MAX_MUTEXES];
static struct mutex mutex_table_lock = MUTEX_INITIALIZER;
```

```
// 뮤텍스 획득 추적
static void track_mutex_acquire(struct mutex *m, const char *file, int line)
{
uint32_t thread_id = thread_get_id();
uint64_t current_time = get_time_us();
mutex_lock(&mutex_table_lock);
for (int i = 0; i < MAX_MUTEXES; i++) {
if (mutex_table[i].mutex == NULL) {
mutex_table[i].mutex = m;
mutex_table[i].owner_thread = thread_id;
mutex_table[i].file = file;
mutex_table[i].line = line;
mutex_table[i].acquire_time = current_time;
break;
}
}
mutex_unlock(&mutex_table_lock);
}


// 데드락 검출
static void detect_deadlock(void)
{
```

```
uint64_t current_time = get_time_us();

mutex_lock(&mutex_table_lock);

for (int i = 0; i < MAX_MUTEXES; i++) {

if (mutex_table[i].mutex != NULL) {

uint64_t hold_time = current_time - mutex_table[i].acquire_time;

if (hold_time > DEADLOCK_TIMEOUT_US) {

EMSG("Potential deadlock detected:");

EMSG(" Thread: %u", mutex_table[i].owner_thread);

EMSG(" Mutex: %p", mutex_table[i].mutex);

EMSG(" Location: %s:%d", mutex_table[i].file, mutex_table[i].line);

EMSG(" Hold time: %llu us", hold_time);

}

}

}

mutex_unlock(&mutex_table_lock);

}



// 매크로로 뮤텍스 추적 자동화

#define TRACKED_MUTEX_LOCK(m) \

do { \

track_mutex_acquire(m, __FILE__, __LINE__); \

mutex_lock(m); \
```

```
} while(0)
```

5.2 레이스 컨디션 검출

```
// 공유 데이터 접근 추적
struct shared_data_access {
void *addr;
size_t size;
uint32_t thread_id;
bool is_write;
uint64_t timestamp;
const char *file;
int line;
};

static struct shared_data_access access_log[MAX_ACCESS_LOG];
static int access_log_index = 0;
static struct mutex access_log_lock = MUTEX_INITIALIZER;

// 메모리 접근 기록
static void log_memory_access(void *addr, size_t size, bool is_write,
const char *file, int line)
{
```

```
uint32_t thread_id = thread_get_id();

uint64_t timestamp = get_time_us();

mutex_lock(&access_log_lock);

int index = access_log_index % MAX_ACCESS_LOG;

access_log[index].addr = addr;

access_log[index].size = size;

access_log[index].thread_id = thread_id;

access_log[index].is_write = is_write;

access_log[index].timestamp = timestamp;

access_log[index].file = file;

access_log[index].line = line;

access_log_index++;

mutex_unlock(&access_log_lock);

// 레이스 컨디션 검사

check_race_condition(addr, size, is_write, thread_id, timestamp);

}



// 레이스 컨디션 검사

static void check_race_condition(void *addr, size_t size, bool is_write,

uint32_t thread_id, uint64_t timestamp)

{

uintptr_t start_addr = (uintptr_t)addr;

uintptr_t end_addr = start_addr + size;
```

```
// 최근 접근 기록 검사

for (int i = 0; i < MAX_ACCESS_LOG; i++) {

struct shared_data_access *access = &access_log[i];

if (access->thread_id == thread_id || access->addr == NULL) {

continue; // 같은 스레드이거나 빈 엔트리

}

uintptr_t other_start = (uintptr_t)access->addr;

uintptr_t other_end = other_start + access->size;

// 메모리 영역 겹침 검사

if (start_addr < other_end && end_addr > other_start) {

// 시간 차이 검사 (레이스 컨디션 가능성)

uint64_t time_diff = timestamp - access->timestamp;

if (time_diff < RACE_CONDITION_WINDOW_US &&

(is_write || access->is_write)) {

EMSG("Race condition detected:");

EMSG(" Address range: %p - %p", (void *)start_addr, (void *)end_addr);

EMSG(" Thread 1: %u (%s) at %s:%d",

access->thread_id, access->is_write ? "write" : "read",

access->file, access->line);

EMSG(" Thread 2: %u (%s) at %s:%d",

thread_id, is_write ? "write" : "read",

__FILE__, __LINE__);

EMSG(" Time difference: %llu us", time_diff);
```

```
}

}

}

}


// 메모리 접근 추적 매크로

#define TRACKED_READ(addr, size) \

log_memory_access(addr, size, false, __FILE__, __LINE__)


#define TRACKED_WRITE(addr, size) \

log_memory_access(addr, size, true, __FILE__, __LINE__)
```

6. 하드웨어 관련 문제

6.1 TrustZone 설정 문제

문제: Secure/Non-secure 메모리 접근 위반

```
// 메모리 보안 상태 검사

static bool is_secure_memory(paddr_t pa)

{

// TZASC (TrustZone Address Space Controller) 설정 확인

uint32_t tzasc_region = (pa >> TZASC_REGION_SHIFT) & TZASC_REGION_MASK;

uint32_t tzasc_config = read_tzasc_region_config(tzasc_region);
```

```
return (tzasc_config & TZASC_SECURE_ACCESS) != 0;

}



// 메모리 접근 권한 검증

static TEE_Result verify_memory_access(vaddr_t va, size_t size, uint32_t flags)

{

paddr_t pa = 0;

// 가상 주소를 물리 주소로 변환

if (core_va2pa(va, &pa) != TEE_SUCCESS) {

EMSG("VA to PA translation failed: 0x%lx", va);

return TEE_ERROR_ACCESS_DENIED;

}

// 보안 메모리 접근 검사

if (!is_secure_memory(pa) && (flags & MEM_ACCESS_SECURE)) {

EMSG("Secure access to non-secure memory: PA 0x%lx", pa);

return TEE_ERROR_ACCESS_DENIED;

}

// 캐시 일관성 검사

if (flags & MEM_ACCESS_CACHED) {

if (!is_memory_cacheable(pa)) {

EMSG("Cached access to non-cacheable memory: PA 0x%lx", pa);

return TEE_ERROR_BAD_PARAMETERS;

}
```

```
}

return TEE_SUCCESS;

}
```

6.2 인터럽트 처리 문제

```
// 인터럽트 통계 추적

struct interrupt_stats {

uint32_t irq_number;

uint64_t count;

uint64_t total_time;

uint64_t max_time;

uint64_t last_timestamp;

};


static struct interrupt_stats irq_stats[MAX_IRQ_NUM];


// 인터럽트 핸들러 래퍼

static void tracked_irq_handler(size_t it, void *data)

{

uint64_t start_time = get_time_us();

// 원래 핸들러 호출

itr_handler_t original_handler = (itr_handler_t)data;
```

```
original_handler(it, NULL);

uint64_t end_time = get_time_us();

uint64_t elapsed = end_time - start_time;

// 통계 업데이트

if (it < MAX_IRQ_NUM) {

irq_stats[it].irq_number = it;

irq_stats[it].count++;

irq_stats[it].total_time += elapsed;

irq_stats[it].last_timestamp = end_time;

if (elapsed > irq_stats[it].max_time) {

irq_stats[it].max_time = elapsed;

}

// 긴 인터럽트 처리 시간 경고

if (elapsed > IRQ_LONG_HANDLER_THRESHOLD) {

EMSG("Long IRQ handler: IRQ %zu took %llu us", it, elapsed);

}

}

}



// 인터럽트 통계 출력

static void dump_irq_stats(void)

{

DMSG("=== Interrupt Statistics ===");
```

```
for (int i = 0; i < MAX_IRQ_NUM; i++) {

if (irq_stats[i].count > 0) {

uint64_t avg_time = irq_stats[i].total_time / irq_stats[i].count;

DMSG("IRQ %u:", irq_stats[i].irq_number);

DMSG(" Count: %llu", irq_stats[i].count);

DMSG(" Total time: %llu us", irq_stats[i].total_time);

DMSG(" Average time: %llu us", avg_time);

DMSG(" Max time: %llu us", irq_stats[i].max_time);

}

}

}
```

7. 로깅 및 디버그 출력

7.1 구조화된 로깅 시스템

```
// 로그 레벨 정의

typedef enum {

LOG_LEVEL_ERROR = 0,

LOG_LEVEL_WARN = 1,

LOG_LEVEL_INFO = 2,

LOG_LEVEL_DEBUG = 3,

LOG_LEVEL_TRACE = 4
```

```
} log_level_t;


// 로그 엔트리 구조체
struct log_entry {
uint64_t timestamp;
uint32_t thread_id;
log_level_t level;
const char *file;
int line;
const char *function;
char message[MAX_LOG_MESSAGE_SIZE];
};


// 순환 로그 버퍼
static struct log_entry log_buffer[MAX_LOG_ENTRIES];
static int log_write_index = 0;
static int log_read_index = 0;
static struct mutex log_mutex = MUTEX_INITIALIZER;


// 구조화된 로그 함수
static void structured_log(log_level_t level, const char *file, int line,
const char *function, const char *format, ...)
{
```

```
    va_list args;

    struct log_entry *entry = NULL;

    uint64_t timestamp = get_time_us();

    uint32_t thread_id = thread_get_id();

    mutex_lock(&log_mutex);

    // 로그 엔트리 할당

    entry = &log_buffer[log_write_index % MAX_LOG_ENTRIES];

    log_write_index++;

    // 버퍼 오버플로우 처리

    if (log_write_index - log_read_index > MAX_LOG_ENTRIES) {

    log_read_index = log_write_index - MAX_LOG_ENTRIES;

    }

    // 로그 엔트리 채우기

    entry->timestamp = timestamp;

    entry->thread_id = thread_id;

    entry->level = level;

    entry->file = file;

    entry->line = line;

    entry->function = function;

    va_start(args, format);

    vsnprintf(entry->message, sizeof(entry->message), format, args);

    va_end(args);

    mutex_unlock(&log_mutex);
```

```
// 즉시 출력 (에러 레벨)
if (level <= LOG_LEVEL_ERROR) {
printf("[%llu] ERROR %s:%d %s(): %s\n",
timestamp, file, line, function, entry->message);
}
}

// 로그 매크로
#define LOG_ERROR(fmt, ...) \
structured_log(LOG_LEVEL_ERROR, __FILE__, __LINE__, __func__, fmt, ##__VA_ARGS__

#define LOG_WARN(fmt, ...) \
structured_log(LOG_LEVEL_WARN, __FILE__, __LINE__, __func__, fmt, ##__VA_ARGS__)

#define LOG_INFO(fmt, ...) \
structured_log(LOG_LEVEL_INFO, __FILE__, __LINE__, __func__, fmt, ##__VA_ARGS__)

#define LOG_DEBUG(fmt, ...) \
structured_log(LOG_LEVEL_DEBUG, __FILE__, __LINE__, __func__, fmt, ##__VA_ARGS__

#define LOG_TRACE(fmt, ...) \
structured_log(LOG_LEVEL_TRACE, __FILE__, __LINE__, __func__, fmt, ##__VA_ARGS__
```

7.2 디버그 덤프 기능

```
// 시스템 상태 덤프
static void dump_system_state(void)
{
DMSG("=== System State Dump ===");
// 스레드 상태
DMSG("Thread Information:");
for (size_t i = 0; i < thread_count; i++) {
struct thread_ctx *thr = &threads[i];
DMSG(" Thread %zu: state=%d, stack_va_end=0x%lx",
i, thr->state, thr->stack_va_end);
}
// 메모리 상태
DMSG("Memory Information:");
dump_heap_stats();
dump_mmap_table();
// 암호화 상태
DMSG("Crypto Information:");
dump_crypto_states();
// 성능 통계
dump_perf_stats();
// 인터럽트 통계
```

```
    dump_irq_stats();

    }



    // 크래시 덤프

    static void crash_dump(uint32_t panic_code, vaddr_t fault_addr)

    {

    EMSG("=== CRASH DUMP ===");

    EMSG("Panic code: 0x%x", panic_code);

    EMSG("Fault address: 0x%lx", fault_addr);

    // 레지스터 덤프

    dump_registers();

    // 스택 트레이스

    dump_stack_trace();

    // 메모리 맵

    dump_memory_map();

    // 최근 로그 엔트리

    dump_recent_logs();

    EMSG("=== END CRASH DUMP ===");

    }



    // 스택 트레이스

    static void dump_stack_trace(void)

    {
```

```
void *stack_frames[MAX_STACK_FRAMES];

int frame_count = 0;

EMSG("Stack trace:");

// 스택 프레임 수집 (플랫폼별 구현 필요)

frame_count = collect_stack_frames(stack_frames, MAX_STACK_FRAMES);

for (int i = 0; i < frame_count; i++) {

const char *symbol = resolve_symbol(stack_frames[i]);

EMSG(" [%d] %p %s", i, stack_frames[i],

symbol ? symbol : "<unknown>");

}

}
```

8. 자동화된 테스트 및 검증

8.1 단위 테스트 프레임워크

```
// 테스트 결과 구조체

struct test_result {

const char *test_name;

bool passed;

const char *error_message;

uint64_t execution_time;

};
```

```
// 테스트 통계
struct test_stats {
int total_tests;
int passed_tests;
int failed_tests;
uint64_t total_time;
};

static struct test_stats global_test_stats = {0};

// 테스트 매크로
#define TEST_ASSERT(condition, message) \
do { \
if (!(condition)) { \
LOG_ERROR("Test assertion failed: %s", message); \
return false; \
} \
} while(0)

#define TEST_ASSERT_EQUAL(expected, actual, message) \
do { \
if ((expected) != (actual)) { \
```

```
LOG_ERROR("Test assertion failed: %s (expected: %d, actual: %d)", \
message, (int)(expected), (int)(actual)); \
return false; \
} \
} while(0)


// 테스트 실행 함수
static bool run_test(const char *test_name, bool (*test_func)(void))
{
uint64_t start_time = get_time_us();
bool result = false;
LOG_INFO("Running test: %s", test_name);
// 테스트 실행
result = test_func();
uint64_t end_time = get_time_us();
uint64_t execution_time = end_time - start_time;
// 통계 업데이트
global_test_stats.total_tests++;
global_test_stats.total_time += execution_time;
if (result) {
global_test_stats.passed_tests++;
LOG_INFO("Test PASSED: %s (%llu us)", test_name, execution_time);
} else {
```

```
    global_test_stats.failed_tests++;

    LOG_ERROR("Test FAILED: %s (%llu us)", test_name, execution_time);

    }

    return result;

    }



    // 테스트 스위트 실행

    static void run_test_suite(void)

    {

    LOG_INFO("=== Starting Test Suite ===");

    // 메모리 관리 테스트

    run_test("memory_allocation", test_memory_allocation);

    run_test("memory_alignment", test_memory_alignment);

    run_test("memory_leak_detection", test_memory_leak_detection);

    // 암호화 테스트

    run_test("aes_encryption", test_aes_encryption);

    run_test("rsa_signature", test_rsa_signature);

    run_test("random_generation", test_random_generation);

    // 동시성 테스트

    run_test("mutex_basic", test_mutex_basic);

    run_test("deadlock_detection", test_deadlock_detection);

    run_test("race_condition", test_race_condition);

    // 결과 출력
```

```
LOG_INFO("=== Test Suite Results ===");

LOG_INFO("Total tests: %d", global_test_stats.total_tests);

LOG_INFO("Passed: %d", global_test_stats.passed_tests);

LOG_INFO("Failed: %d", global_test_stats.failed_tests);

LOG_INFO("Total time: %llu us", global_test_stats.total_time);

if (global_test_stats.failed_tests > 0) {

LOG_ERROR("Some tests failed!");

} else {

LOG_INFO("All tests passed!");

}

}
```

8.2 스트레스 테스트

```
// 스트레스 테스트 설정

struct stress_test_config {

int num_threads;

int duration_seconds;

int operations_per_second;

bool enable_memory_stress;

bool enable_crypto_stress;

bool enable_io_stress;

};
```

```
// 스트레스 테스트 실행
static void run_stress_test(struct stress_test_config *config)
{
LOG_INFO("=== Starting Stress Test ===");
LOG_INFO("Threads: %d", config->num_threads);
LOG_INFO("Duration: %d seconds", config->duration_seconds);
LOG_INFO("Operations/sec: %d", config->operations_per_second);
uint64_t start_time = get_time_us();
uint64_t end_time = start_time + (config->duration_seconds * 1000000ULL);
// 스트레스 테스트 스레드 생성 (의사 코드)
for (int i = 0; i < config->num_threads; i++) {
create_stress_thread(i, config, start_time, end_time);
}
// 테스트 완료 대기
wait_for_stress_threads();
// 결과 분석
analyze_stress_test_results();
LOG_INFO("=== Stress Test Completed ===");
}


// 메모리 스트레스 테스트
static void memory_stress_worker(void)
```

```
{
const size_t alloc_sizes[] = {64, 256, 1024, 4096, 16384};
const int num_sizes = sizeof(alloc_sizes) / sizeof(alloc_sizes[0]);
void *ptrs[100];
int ptr_count = 0;
while (stress_test_running) {
// 랜덤 할당
if (ptr_count < 100) {
size_t size = alloc_sizes[rand() % num_sizes];
ptrs[ptr_count] = TEE_Malloc(size, 0);
if (ptrs[ptr_count]) {
// 메모리 패턴 쓰기
memset(ptrs[ptr_count], 0xAA, size);
ptr_count++;
}
}
// 랜덤 해제
if (ptr_count > 0 && (rand() % 3) == 0) {
int index = rand() % ptr_count;
TEE_Free(ptrs[index]);
ptrs[index] = ptrs[ptr_count - 1];
ptr_count--;
}
```

```
// CPU 양보

thread_yield();

}

// 정리

for (int i = 0; i < ptr_count; i++) {

TEE_Free(ptrs[i]);

}

}
```

이 디버깅 및 트러블슈팅 가이드는 OP-TEE 개발 과정에서 발생할 수 있는 다양한 문제들과 그 해결 방법을 실용적인 관점에서 제공합니다. 실제 코드 예제와 함께 디버깅 기법들을 설명하여 개발자들이 효율적으로 문제를 해결할 수 있도록 도움을 줍니다.